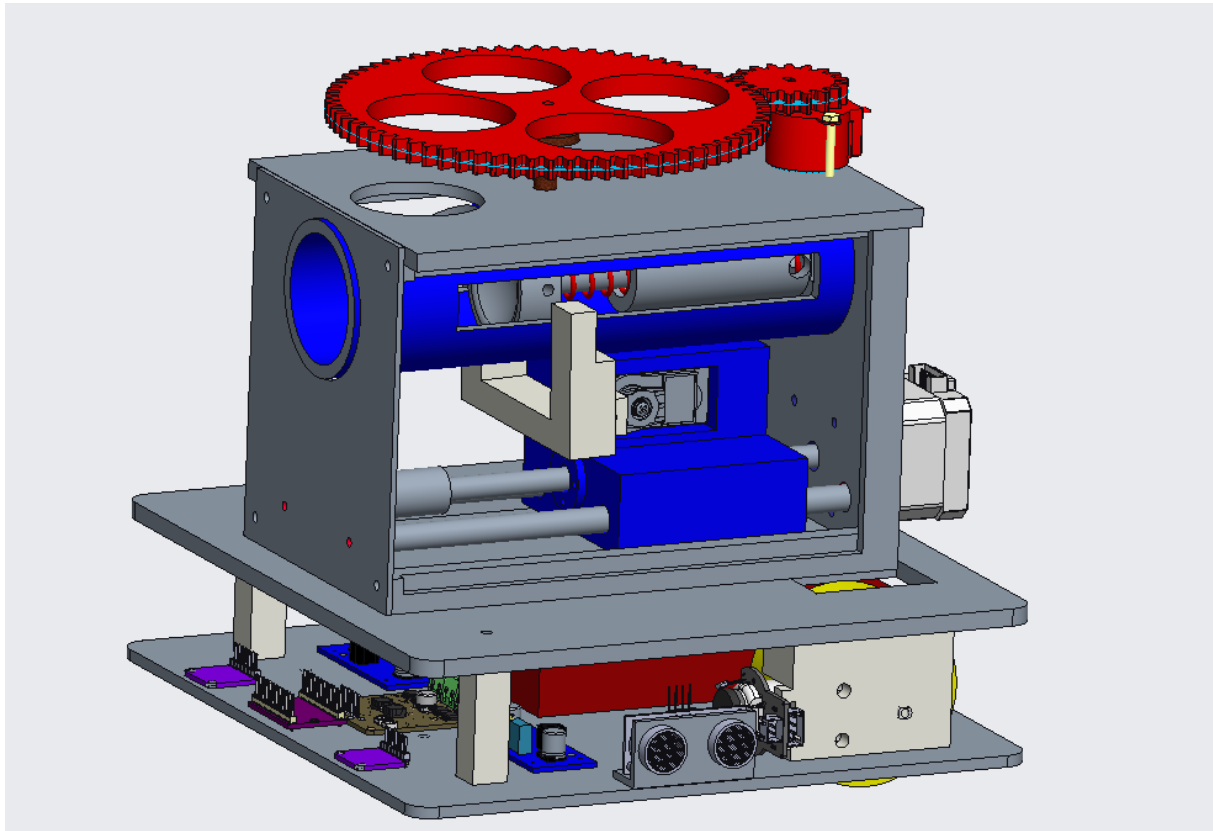




Projet Mécatronique

La Guerre des trônes

Elie Amani, Louis Matter, Luca Beck

4^e Semestre Mécatronique Trinational 2026 – Promotion Kelvin



Mulhouse, 21.05.2026

Sommaire

Introduction	2
1 Choix techniques de conception	2
2 Choix des composants	3
2.1 Le canon	3
2.1.1 1. Stepper moteur NEMA 17 avec Vis T8 intégrée + driver DRV8825	3
2.1.2 2. Servo moteur FS5115M	3
2.1.3 3. Adaptateur pour balle de golf	4
2.1.4 4. Ressort	4
2.1.5 5. Le positionnement du canon : Servo moteur SG90 et capteur ToF VL53L1X	4
2.1.6 6. Le chargeur	5
2.2 Le déplacement	5
2.2.1 1. Moteurs, encodeurs FIT0450 et driver MDD3A	5
2.2.2 2. Détection des robots adverses	6
2.2.3 3. Détection de l'eau et du pont à l'aide de capteurs de couleur . . .	6
2.3 Programmation	6
2.4 Alimentation	7
3 Méthode de programmation	10
3.1 Le tir d'obus	10
3.1.1 1. Le rechargement d'obus	10
3.1.2 2. La gâchette	11
3.1.3 3. Le stepper moteur avec la vis sans fin	12
3.2 Le déplacement du robot	15
3.3 La machine à états	15
4 Tableau des coûts	18
Conclusion	19

Introduction

Dans le cadre du projet de mécatronique du 4^e semestre, notre équipe doit concevoir un robot autonome capable d'évoluer sur une table de dimensions 2000×900 mm. Deux châteaux, constitués de colonnes de canettes, sont placés sur les rives opposées d'une rivière reliées par un pont de seulement 20 cm de large.

Les objectifs du projet sont les suivants :

- Détruire le château adverse à l'aide de tirs d'obus (balles de golf), chaque tir ne devant pas dépasser une énergie de 1 joule.
- Ou traverser le pont afin de renverser directement les canettes une fois sur la rive opposée.

Le cahier des charges impose également plusieurs contraintes importantes :

- Dimensions maximales du robot au départ : $250 \text{ mm} \times 250 \text{ mm} \times 250 \text{ mm}$.
- Fonctionnement entièrement autonome pendant une durée de 3 minutes.
- Respect d'un budget maximal de 150 € par équipe.
- Gestion des interactions possibles avec l'autre robot (croisement sur le pont, blocage, évitement, etc.).

Ce document présente les choix techniques retenus, la sélection des composants, la stratégie de programmation ainsi que le bilan financier de notre solution.

Choix techniques de conception

Notre robot fonctionne selon un principe de tir par ressort avec un système de rechargement automatique. Notre objectif principal est de détruire le château adverse à distance grâce au canon, avant même d'envisager une traversée du pont, qui représente une manœuvre plus risquée. La stratégie prévue consiste donc à faire avancer légèrement le robot, le pivoter de 90° , puis à le déplacer jusqu'à une position de tir située à proximité du pont afin d'effectuer les tirs depuis cet emplacement. Notre objectif secondaire est de traverser le pont afin de renverser directement les canettes adverses.

Concernant l'assemblage mécanique, le robot est constitué d'une plateforme principale sur laquelle est monté le canon, ainsi que d'une plateforme secondaire placée en dessous afin de laisser suffisamment d'espace pour les composants électroniques, les moteurs, les roues et le câblage reliant les cartes Arduino Mega et Arduino Nano.

Le déplacement du robot est assuré par deux roues motrices ainsi qu'une roue folle placée à l'avant du châssis afin d'améliorer la stabilité. Plusieurs capteurs sont également répartis autour du robot afin de détecter les changements de couleur sur la table (rivière, pont, bordures), mesurer les distances aux obstacles à l'aide de capteurs ultrasoniques et détecter les canettes grâce à un capteur ToF (Time of Flight).

Le système de tir du canon repose sur une gâchette imprimée en 3D à partir de notre propre conception. Cette gâchette est actionnée par un servo moteur et montée sur un bloc guidé par deux arbres usinés. Le déplacement du système est assuré par une vis sans fin contrôlée par un moteur pas à pas NEMA 17.

À l'intérieur du tube du canon, contenant le ressort ainsi que la balle de golf, nous avons conçu un adaptateur sur lequel repose l'obus. Cet adaptateur coulisse au centre du ressort et est retenu par la gâchette afin de permettre la compression du ressort. Lorsque la gâchette est relâchée, le ressort se détend et propulse l'obus. Ce mécanisme est rendu possible grâce à deux rainures le long du tube du canon permettant le passage d'une tige reliant l'adaptateur au système de gâchette.

Le canon est constitué de deux pièces principales imprimées en 3D : le corps du canon ainsi qu'un couvercle fixé par vis. Cette conception simplifie l'impression et l'assemblage tout en assurant une bonne rigidité mécanique afin de supporter les efforts générés lors des tirs. Le couvercle permet également l'accès aux composants internes lors du montage et maintient en position les deux arbres de guidage du mécanisme.

Enfin, le système de rechargement utilise un mécanisme d'engrenages composé d'une petite roue dentée entraînant une grande roue dentée. Cette réduction mécanique permet d'effectuer le rechargement des obus avec un moteur pas à pas plus compact et plus économique.

Choix des composants

Le canon

1. Stepper moteur NEMA 17 avec Vis T8 intégrée + driver DRV8825

Calcul du couple moteur

On utilise la relation :

$$C = \frac{F_r p}{2\pi n}$$

Avec les valeurs :

$$F_r = 66.66 \text{ N}, \quad p = 0.008 \text{ m}, \quad n = 0.35$$

Substitution :

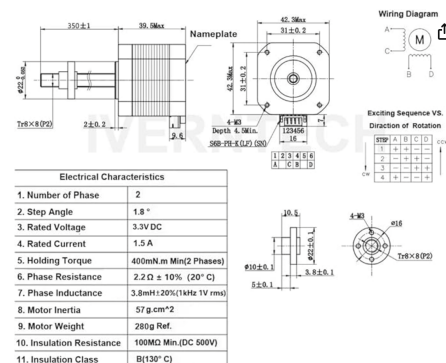
$$C = \frac{66.66 \times 0.008}{2\pi \times 0.35}$$

Calcul numérique :

$$C = 0.24 \text{ N} \cdot \text{m}$$

Conversion :

$$C = 240 \text{ mN} \cdot \text{m}$$



Le principe de fonctionnement de notre canon repose sur la compression d'un ressort. Pour comprimer ce ressort, nous utilisons un moteur pas à pas NEMA 17 associé à une vis sans fin. Un bloc mobile est vissé sur cette vis sans fin afin de transformer le mouvement de rotation du moteur en un déplacement linéaire du bloc. Lorsque le moteur tourne, la vis sans fin entraîne le bloc vers l'arrière, ce qui permet de reculer la gâchette et de comprimer progressivement le ressort du canon.

2. Servo moteur FS5115M

Le servo moteur FS5115M est intégré directement dans le bloc déplacé par la vis sans fin commandée par le NEMA 17. Une gâchette imprimée en 3D est fixée sur l'axe du servo moteur afin de contrôler mécaniquement le maintien ou le relâchement du ressort.

Lorsque le ressort est complètement comprimé, le servo moteur maintient la gâchette en position afin de retenir le mécanisme. Une fois la compression terminée, le servo change de position et relâche la gâchette, ce qui libère le ressort et provoque le tir de l'obus.

3. Adaptateur pour balle de golf

L'adaptateur servant à propulser la balle de golf est une pièce en aluminium que nous avons usinée en FAO. Cette pièce permet d'assurer la liaison mécanique entre la gâchette et la balle de golf.

L'adaptateur est positionné à l'intérieur du tube du canon et coulisse le long de celui-ci lors de la compression et du relâchement du ressort. Lorsque le ressort est comprimé, l'adaptateur est retenu par la gâchette. Une fois la gâchette relâchée, le ressort pousse rapidement l'adaptateur vers l'avant, ce qui transmet l'énergie mécanique à la balle de golf afin de réaliser le tir.

L'utilisation de l'aluminium permet d'obtenir une pièce suffisamment résistante tout en conservant une masse relativement faible afin de limiter les pertes d'énergie durant le tir.

4. Ressort

Calcul de la raideur du ressort

On utilise la loi de Hooke :

$$F_r = k x$$

La compression imposée est :

$$x = 3 \text{ cm} = 0.03 \text{ m}$$

L'énergie potentielle élastique doit rester inférieure à 1 J :

$$E = \frac{1}{2} k x^2 < 1$$

En remplaçant $x = 0.03 \text{ m}$:

$$\frac{1}{2} k \cdot (0.03)^2 < 1$$

On isole k :

$$k < \frac{2}{(0.03)^2}$$

Calcul numérique :

$$k < 2222 \text{ N/m}$$

Conversion :

$$k < 2.2 \text{ N/mm}$$

Le fonctionnement du canon reposant entièrement sur l'énergie stockée dans un ressort, nous avons dû effectuer plusieurs calculs afin de sélectionner un ressort adapté tout en respectant la contrainte imposée de ne pas dépasser une énergie de 1 joule lors du tir.

5. Le positionnement du canon : Servo moteur SG90 et capteur ToF VL53L1X

Afin de pouvoir viser les canettes avant d'effectuer un tir, notre robot utilise un capteur ToF (Time of Flight) VL53L1X. Ce capteur permet de mesurer précisément une distance en envoyant une impulsion lumineuse infrarouge puis en calculant le temps nécessaire pour que cette lumière revienne après réflexion sur un objet. Plus le temps mesuré est grand,

plus l'objet détecté est éloigné.

Contrairement à un capteur ultrasonique, dont la zone de détection est relativement large et conique, le capteur ToF utilise un faisceau lumineux beaucoup plus précis. Cela permet d'obtenir une meilleure localisation des canettes et d'éviter des détections imprécises.

Le capteur ToF est monté sur un servo moteur SG90 afin de pouvoir effectuer une rotation et scanner la zone située devant le robot. Pendant ce balayage, le programme mesure continuellement les distances détectées par le capteur. Lorsqu'une canette est détectée, l'angle actuel du servo moteur est enregistré.

Le robot peut ensuite utiliser cette information pour se réorienter et placer le canon dans le même axe que la canette détectée avant de tirer.

Nous avons également pris soin de positionner le capteur ToF suffisamment en hauteur afin d'éviter qu'il détecte les balles de golf présentes sur la table à la place des canettes.

6. Le chargeur

Le système de chargement des obus est assuré par un mécanisme d'engrenages entraîné par un moteur pas à pas 28BYJ-48 piloté par un driver de type ULN2003. Le moteur entraîne une roue dentée dans laquelle nous avons conçu plusieurs emplacements permettant de maintenir les balles de golf.

Après chaque tir, le moteur pas à pas fait tourner la roue dentée afin d'aligner automatiquement la balle suivante avec l'entrée du canon. Ce système permet donc un rechargement entièrement automatisé sans intervention extérieure.

Le mécanisme utilise une réduction par engrenages afin d'obtenir un mouvement plus précis et suffisamment de couple malgré l'utilisation d'un moteur relativement compact et économique.

Le déplacement

1. Moteurs, encodeurs FIT0450 et driver MDD3A

Le déplacement du robot est assuré par deux moteurs DC équipés d'encodeurs FIT0450 avec un rapport de réduction de 120:1. Ces ensembles moteur + encodeur ont été achetés chez GO TRONIC dans le but de permettre un déplacement précis du robot grâce à l'odométrie.

Les encodeurs permettent de mesurer le nombre de rotations effectuées par les roues. À partir de ces informations, il est possible d'estimer :

- La distance parcourue par le robot,
- Sa vitesse,
- Ainsi que son angle de rotation.

L'odométrie permet donc de suivre approximativement la position du robot au cours du temps durant le parcours.

Grâce à ces mesures, nous avons pu programmer différentes fonctions de déplacement telles que :

- avancer d'une certaine distance à une vitesse donnée,

- maintenir une trajectoire rectiligne,
- ou encore effectuer une rotation d'un angle précis.

Les moteurs sont contrôlés par un driver moteur MDD3A permettant d'alimenter les deux moteurs DC et de contrôler leur vitesse ainsi que leur sens de rotation.

2. Détection des robots adverses

Le capteur ToF utilisé pour le positionnement du canon est également utilisé pour détecter la présence d'un robot adverse devant notre robot.

Le programme surveille en permanence la distance mesurée par le capteur. Si un obstacle est détecté à une distance inférieure à environ 10 cm, le robot peut interrompre son déplacement afin d'éviter une collision avec le robot adverse.

Cette détection permet d'améliorer la sécurité du robot ainsi que sa capacité à réagir face aux interactions possibles durant le défi.

3. Détection de l'eau et du pont à l'aide de capteurs de couleur

Le robot utilise deux capteurs de couleur DFROBOT afin de détecter les différentes zones de la table, notamment :

- Le pont,
- La rivière

Les deux capteurs sont placés sous le robot avec un écart légèrement supérieur à l'entraxe des roues, mais inférieur à la largeur du pont.

Cette disposition permet de vérifier que les deux capteurs détectent simultanément la couleur du pont pendant la traversée. Si l'un des capteurs ne détecte plus la bonne couleur, cela signifie que le robot commence à sortir du pont. Le programme peut alors corriger automatiquement sa trajectoire afin de se repositionner correctement.

Programmation

Pour la partie programmation, nous utilisons une carte Arduino Mega 2560 comme contrôleur principal du robot.

Le choix de l'Arduino Mega s'explique par le grand nombre d'entrées/sorties nécessaires au fonctionnement du système :

- 16 pins digitales,
- 8 pins PWM,
- 2 pins dédiées à la communication UART avec l'Arduino Nano,
- ainsi que 4 pins utilisant les interruptions pour la lecture des encodeurs des moteurs.

Nous utilisons également un Arduino Nano en complément de l'Arduino Mega. Cette solution a été mise en place à cause de problèmes de conflits entre le servo moteur contrôlant la rotation du capteur ToF et la communication I2C utilisée par ce même capteur.

En effet, le capteur ToF VL53L1X communique via le protocole I2C, tandis que le servo moteur nécessite un signal PWM stable. Lors de nos essais, nous avons observé que les pics de courant du servo moteur avec ces signaux PWM perturbait la communication I2C qu'utilise notre capteur ToF qui rendait donc des valeurs inutilisables.

Afin d'éviter ces conflits, le capteur ToF a été connecté à un Arduino Nano séparé. Les données mesurées par le capteur sont ensuite transmises à l'Arduino Mega grâce à une communication UART utilisant les broches RX et TX.

Cette séparation des tâches permet d'obtenir un fonctionnement plus stable du système global.

Alimentation

Pour alimenter le robot, nous utilisons une batterie LiPo 3S de 5000 mAh et 50C récupérée d'une voiture télécommandée.

Cette batterie fournit une tension d'environ 12 V. Afin d'alimenter correctement les différents composants du robot, nous utilisons deux convertisseurs abaisseurs de tension (*step-down converters*) :

- Un convertisseur $12\text{ V} \rightarrow 5\text{ V}$ destiné aux cartes Arduino ainsi qu'aux capteurs,
- Un convertisseur $12\text{ V} \rightarrow 7\text{ V}$ destiné aux moteurs DC.

Cette séparation des alimentations permet d'assurer une tension adaptée à chaque composant tout en limitant les perturbations électriques entre les moteurs et l'électronique de contrôle.

Ci-dessous nos branchements électriques :

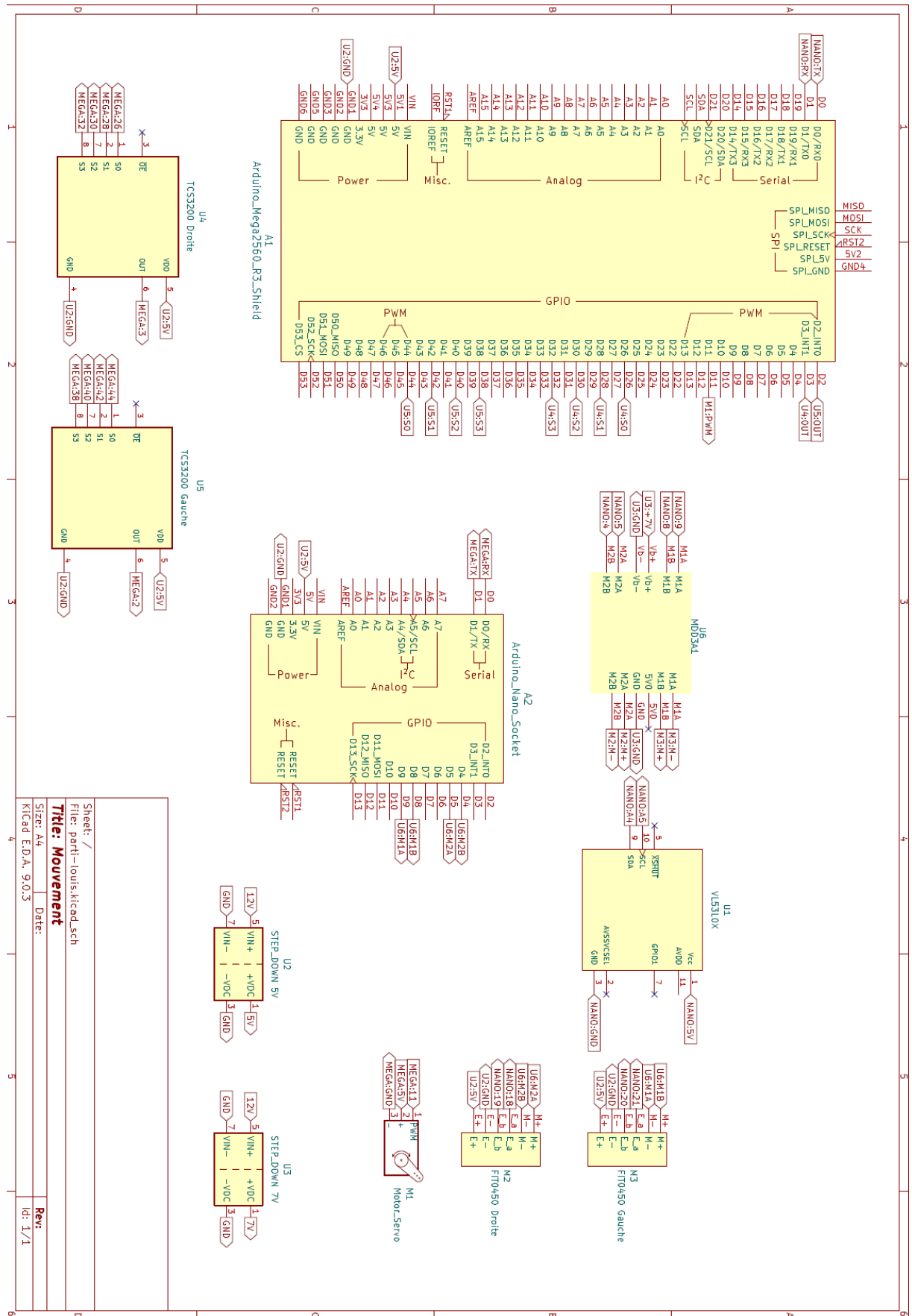


Figure 1: Les branchements nécessaires au déplacement du robot.

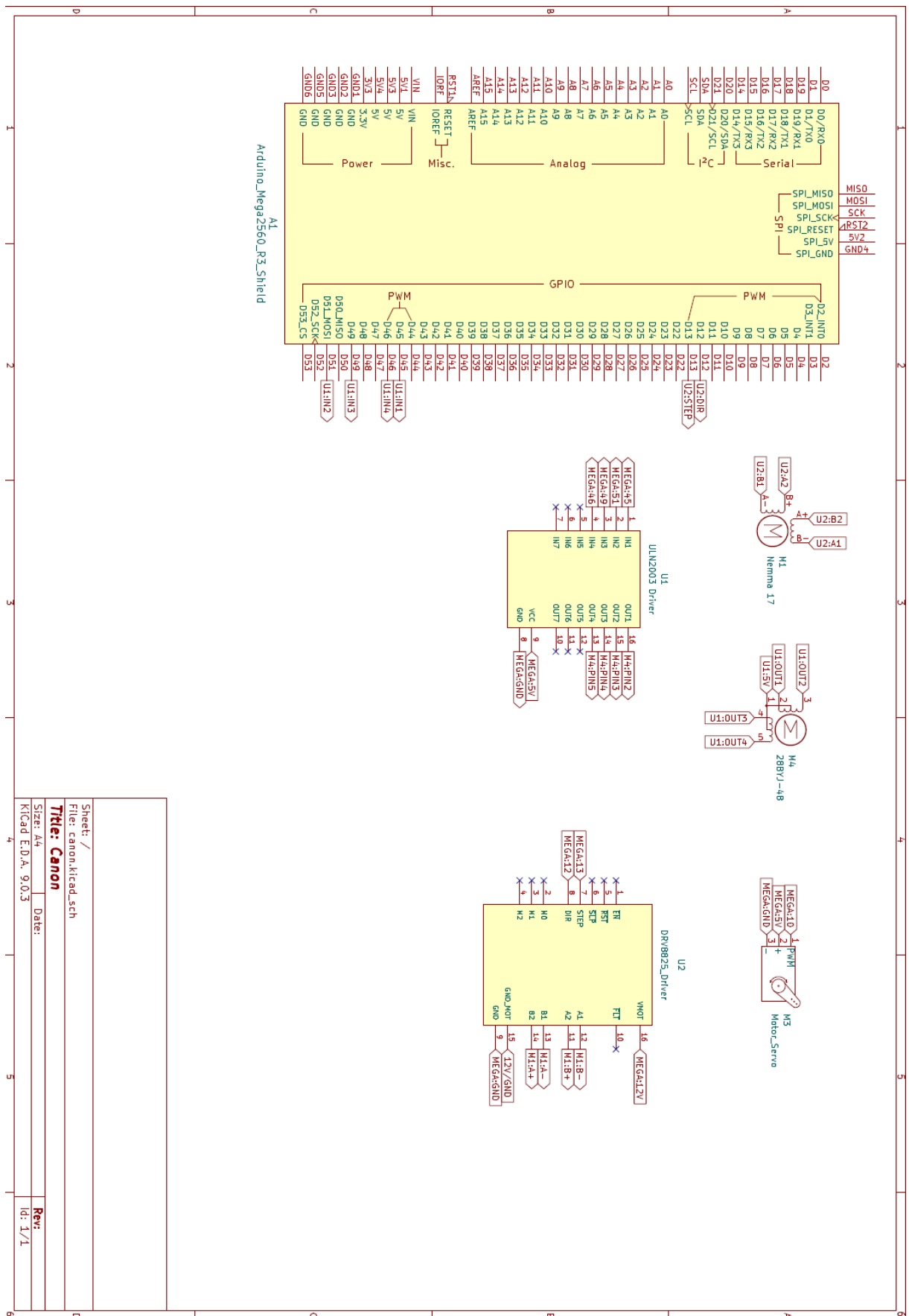


Figure 2: Les branchements nécessaires pour le canon.

Méthode de programmation

Afin de faire fonctionner le robot, nous utilisons une carte Arduino Mega, que nous avons codé dans l'IDE Arduino.

Le tir d'obus

La programmation du tir d'obus a été décomposée en plusieurs parties : le rechargement d'obus, le fonctionnement de la gâchette et l'utilisation du stepper moteur pour actionner la vis sans fin.

1. Le rechargement d'obus

Comme nous l'avons expliqué précédemment, le rechargement des obus fonctionne grâce à un système d'engrenages relié à un moteur pas à pas 28BYJ-48 contrôlé à l'aide de la librairie AccelStepper. Ce moteur entraîne une roue dentée permettant de faire tourner le chargeur afin d'amener un nouvel obus au-dessus du canon après chaque tir.

Le système d'engrenage possède un rapport de transmission défini par :

$$\text{rapport} = \frac{\text{Dents sortie}}{\text{Dents entrée}} = \frac{73}{18} \approx 4,0556 \quad (1)$$

Le rechargeur doit effectuer une rotation de 90, soit un quart de tour complet :

$$90 = \frac{1}{4} \times 360 \quad (2)$$

Le moteur 28BYJ-48 est utilisé en mode *half-step* avec la librairie AccelStepper. Dans cette configuration, le moteur nécessite environ 4096 demi-pas pour effectuer une rotation complète de 360.

Le nombre de pas nécessaires pour faire tourner la roue dentée de 90 devient donc :

$$4,0556 \times \frac{1}{4} \times 4096 \approx 4154 \text{ pas} \quad (3)$$

Cependant, après plusieurs essais expérimentaux sur le système réel, une valeur légèrement ajustée de 4140 pas a été retenue afin d'obtenir un alignement mécanique plus précis du chargeur avec le canon.

Dans le programme, cette valeur est stockée dans la constante :

```
//distance de rotation rechargeur  
const long STEPS_RECHARGE = 4140;
```

Le moteur rechargeur est ensuite configuré grâce à la librairie AccelStepper :

```
AccelStepper rechargeur(  
  HALFSTEP,  
  IN1, IN3, IN2, IN4  
);
```

Le mode HALFSTEP permet un mouvement plus fluide et plus précis qu’avec la librairie Stepper utilisée initialement. Cette méthode réduit également les vibrations et améliore le positionnement du chargeur.

La vitesse maximale ainsi que l’accélération du moteur sont définies dans le `setup()` :

```
rechargeur.setMaxSpeed(1600);  
rechargeur.setAcceleration(1000);
```

Lorsqu’un tir est terminé, le programme passe à l’état RECHARGE. Le moteur effectue alors une rotation de 4140 pas afin de positionner l’obus suivant devant le canon :

```
//on recharge avant prochain tir  
rechargeur.move(STEPS_RECHARGE);
```

Le moteur continue ensuite son déplacement grâce à :

```
rechargeur.run();
```

Cette fonction est appelée continuellement dans la boucle principale `loop()`, ce qui permet d’obtenir un fonctionnement non bloquant et plus fluide du système global du robot.

2. La gâchette

Le système de gâchette du robot fonctionne à l’aide d’un servo moteur FS5115M contrôlé par la librairie `Servo.h` d’Arduino. Ce servo moteur permet d’actionner mécaniquement la gâchette afin de retenir puis relâcher le ressort responsable du tir de l’obus.

Dans le programme, le servo moteur est déclaré de la manière suivante :

```
Servo FS5115M;
```

Puis relié au port numérique 10 de la carte Arduino :

```
int servoPin = 10;
```

Le servo moteur possède deux positions principales correspondant aux deux états mécaniques de la gâchette :

```
//positions servo
const int SERVO_BAS = 47;
const int SERVO_HAUT = 25;
```

- La position `SERVO_BAS` correspond à la gâchette relâchée.
- La position `SERVO_HAUT` correspond à la gâchette levée afin de retenir le ressort pendant sa compression.

Le servo est initialisé dans la fonction `setup()` grâce à :

```
FS5115M.attach(servoPin);
```

Au démarrage du programme, la gâchette est placée directement en position basse :

```
//gachette basse au début
FS5115M.write(SERVO_BAS);
```

Le fonctionnement de la gâchette est synchronisé avec la vis sans fin et la machine à états du programme principal.

Durant la phase `TIR_AVANCE`, la vis sans fin avance afin de déplacer le mécanisme de compression. Lorsque ce mouvement est terminé, le servo moteur relève la gâchette :

```
//on monte la gachette
FS5115M.write(SERVO_HAUT);
```

Cette action permet de retenir le ressort pendant que la vis sans fin recule afin de le comprimer.

Ensuite, pendant l'état `TIR_RETOUR`, lorsque le recul de la vis sans fin est terminé, la gâchette est relâchée grâce à la remise du servo en position basse.

Le ressort est alors libéré, ce qui provoque le tir de l'obus.

3. Le stepper moteur avec la vis sans fin

Le mécanisme principal permettant de comprimer le ressort du canon utilise un moteur pas à pas NEMA contrôlé par un pilote de type STEP/DIR ainsi qu'une vis sans fin. Ce système transforme le mouvement de rotation du moteur en un déplacement mécanique contrôlé permettant de comprimer puis relâcher le ressort servant au tir de l'obus.

Le moteur est contrôlé à l'aide de la librairie `AccelStepper`, permettant un fonctionnement plus fluide et plus précis qu'une simple génération manuelle d'impulsions électriques.

Dans le programme, le moteur est déclaré de la manière suivante :

```
//initialisation pins stepper vis sans fin
#define STEP_PIN 13
#define DIR_PIN 12

AccelStepper visSansFin(AccelStepper::DRIVER, STEP_PIN, DIR_PIN);
```

Le mode *driver* est utilisé car le moteur est piloté par un contrôleur externe utilisant uniquement :

- une broche *step* pour envoyer les impulsions,
- une broche *dir* pour définir le sens de rotation.

La distance de déplacement de la vis sans fin est définie par :

```
//distance de déplacement de la vis
const long STEPS_VIS = 1290;
```

Cette valeur correspond expérimentalement au nombre de pas nécessaires pour :

- avancer le mécanisme,
- comprimer complètement le ressort,
- puis revenir à sa position initiale.

Le moteur est ensuite configuré dans la fonction `setup()` :

```
visSansFin.setMaxSpeed(700);
visSansFin.setAcceleration(900);
```

- `setMaxSpeed()` définit la vitesse maximale du moteur.
- `setAcceleration()` permet de limiter les changements brusques de vitesse afin de réduire les vibrations et les chocs mécaniques.

Au démarrage du programme, la position actuelle du moteur est définie comme position de référence :

```
visSansFin.setCurrentPosition(0);
```

Le premier déplacement du moteur est ensuite lancé avec :

```
visSansFin.moveTo(-STEPS_VIS);
```

Le signe négatif indique simplement le sens de déplacement choisi pour éloigner le bloc de la butée mécanique.

Pendant l'exécution du programme, la fonction `visSansFin.run()` est appelée continuellement dans la boucle principale `loop()`. Cette méthode permet au moteur de fonctionner de manière non bloquante tout en gérant automatiquement :

- l'accélération,
- la décélération,
- ainsi que le positionnement précis du moteur.

Le fonctionnement de la vis sans fin est divisé en deux phases :

3.1. Avance du mécanisme (TIR_AVANCE) La vis sans fin avance afin de déplacer le système mécanique et préparer la compression du ressort.

Lorsque le déplacement est terminé :

```
//quand avance terminée  
if (visSansFin.distanceToGo() == 0)
```

le servo moteur relève la gâchette afin de verrouiller le ressort.

3.2. Retour du mécanisme (TIR_RETOUR) Le moteur revient ensuite vers sa position initiale :

```
visSansFin.moveTo(0);
```

Pendant ce retour, le ressort est comprimé mécaniquement. Une fois le mouvement terminé, la gâchette est relâchée.

Le déplacement du robot

```
// ===== INTERRUPTIONS =====  
void readEncoderD() {  
    if (digitalRead(ENC2_D) == HIGH) countD++;  
    else countD--;  
}  
  
void readEncoderG() {  
    if (digitalRead(ENC2_G) == HIGH) countG++;  
    else countG--;  
}  
  
// ===== ODOMÉTRIE =====  
void updateOdometrie(long deltaD, long deltaG) {  
    float distD = (deltaD * 2.0 * PI * RAYON_ROUE) / TICKS_PAR_TOUR;  
    float distG = (deltaG * 2.0 * PI * RAYON_ROUE) / TICKS_PAR_TOUR;  
  
    float dC = (distD + distG) / 2.0;  
    float dTheta = (distD - distG) / ENTRE_AXES;  
  
    theta += dTheta;  
    while (theta > PI) theta -= 2*PI;  
    while (theta < -PI) theta += 2*PI;  
    x += dC * cos(theta);  
    y += dC * sin(theta);  
}
```

Avec ce code nous récupérons les interruptions des encodeurs sur nos roues avec les fonctions `readEncoder()` et avec ces valeurs dans la fonction `updateOdometrie()` on recalcule à chaque tour de boucle la position de notre robot et son angle d'inclinaison.

La machine à états

Le déplacement autonome du robot ainsi que l'enchaînement des différentes actions sont gérés à l'aide d'une machine à états programmée sur l'Arduino Mega 2560. Cette méthode permet d'organiser le comportement du robot en plusieurs étapes successives tout en conservant un fonctionnement non bloquant.

Les différents états du robot sont définis dans une énumération :

```
// ===== ETATS ROBOT =====
enum EtatRobot {
    IDLE,
    DEBUT,
    AVANCE_COURT,
    TOURNE_ROUE,
    TOURNE_VERS,
    VA_VERS,
    RAPPROCHEPONT,
    AJUSTE2CM,
    ORIENTATION,
    TOURNER_VERS_PONT,
    AVANCER_VERS_FIN_PONT,
    TOURNER_SUR_PONT,
    AVANCER_SUR_PONTCAPTEUR,
    AVANCER_PONT,
    ORIENTATION_APRES_PONT,
    AVANCER_BOUTEILLE,
    FINI
};
```

Chaque état correspond à une action précise du robot, comme avancer, tourner, détecter une canette, tirer ou traverser le pont.

Le programme principal fonctionne à l'aide d'un **switch-case** exécuté dans la boucle `loop()`. Selon l'état actuel du robot, une fonction spécifique est exécutée puis le programme passe à l'état suivant lorsque l'action est terminée.

1. État IDLE Cet état correspond à l'attente avant le démarrage du robot. Aucun mouvement n'est effectué tant que le programme ne passe pas à l'état suivant.

2. État DEBUT Cet état marque le début du parcours autonome. Le robot initialise sa séquence de déplacement avant de commencer à se positionner sur la table.

3. État AVANCE_COURT Dans cette phase, le robot avance légèrement afin de quitter sa position initiale et préparer son orientation vers la zone de tir.

Le déplacement est réalisé grâce à la fonction `avancerDroit` qui utilise l'odométrie et les encodeurs pour avancer d'une distance précise.

4. États TOURNE_ROUE et TOURNE_VERS Ces états permettent au robot d'effectuer des rotations contrôlées.

La fonction `tournerVersNonBloquant()` utilise l'angle calculé par l'odométrie afin d'orienter précisément le robot dans la direction souhaitée.

Une version utilisant une seule roue est également utilisée : `tournerUneRoueNonBloquant()` afin de réaliser certaines corrections de trajectoire plus précises.

5. États VA_VERS, RAPPROCHEPONT et AJUSTE2CM Ces états servent à déplacer progressivement le robot vers le pont et à ajuster précisément sa position avant la traversée.

Le robot utilise alors :

- l'odométrie,
- les encodeurs,
- ainsi que les capteurs ultrasoniques et ToF.

Cela permet d'obtenir un meilleur positionnement avant les phases critiques du parcours.

6. État ORIENTATION Pendant cette phase, le servo moteur effectue une rotation du capteur ToF afin de scanner la zone des canettes.

Le programme utilise la fonction `ServoMoteur_rotation()` pour faire pivoter le capteur progressivement.

Pendant le balayage, le robot lit continuellement les distances détectées grâce à `Lecture_ToF()`.

Lorsqu'une canette est détectée, l'angle du servo est enregistré afin de permettre au robot de s'orienter correctement vers la cible avant le tir.

7. États TOURNER_VERS_PONT, AVANCER_VERS_FIN_PONT et TOURNER_SUR_PONT Ces états correspondent au positionnement du robot avant la traversée du pont.

Le robot effectue :

- une orientation vers le pont,
- une avancée contrôlée,
- puis un réalignement avant d'entrer sur le pont.

8. États AVANCER_SUR_PONTCAPTEUR et AVANCER_PONT Ces états gèrent la traversée du pont.

Le robot utilise les deux capteurs de couleur afin de vérifier qu'il reste bien aligné sur le pont.

La fonction `avancer_pont_capteur()` permet de corriger automatiquement la trajectoire si un des capteurs ne détecte plus la bonne couleur.

Cette méthode permet de limiter les risques de sortie du pont pendant la traversée.

9. État ORIENTATION_APRES_PONT Une fois le pont traversé, le robot effectue une dernière rotation afin de se diriger vers les canettes adverses.

10. État AVANCER_BOUTEILLE Dans cette dernière phase, le robot avance vers les canettes afin de les renverser directement si nécessaire.

11. État FINI Cet état marque la fin complète du programme autonome. Tous les moteurs sont arrêtés et le robot ne réalise plus d'action.

L'utilisation d'une machine à états permet de structurer clairement le comportement du robot tout en gardant un programme lisible et facilement modifiable. Cette architecture facilite également l'ajout de nouvelles fonctionnalités et la gestion des différentes phases du parcours autonome.

Tableau des coûts

Désignation	Qté	Prix (€)	
Moteur + encodeur FIT0450	2	15,06	
Commande 2x moteur MDD3A	1	9,90	
Moteur NEMA 17 + vis sans fin	1	22,59	
Adaptateur batterie Lipo RC XT60 femelle	1	0,99	
Ligne dupont m-m	1	1,62	
Ligne dupont m-f	1	1,66	
Ligne dupont f-f	1	1,76	
Roues folles CY-12A	3	3,39	
Arduino mega 2560 R3	1	13,29	
Driver moteur de pas à pas DRV8825	1	1,24	
Capteur TOF VL53L1X	1	4,11	
Ressort compression	1	3,99	
Stepper 28BYJ-48	1	0,99	
Servo FS5115M	1	10,19	
Ultrason HC-SR04	1	0,99	
Module d'alimentation 5V/7V 5A XL4015 DC-DC	2	3	
Capteur couleur TCS3200	2	6,99	
Servo SG90	1	0,99	
Batterie 5000mAh LiPo 3S	1	20	
Arduino Nano 3.0	1	0,99	
Total			123,74 €

Table 1: Tableau récapitulatif des coûts du projet.

Conclusion

À travers ce projet, nous avons mobilisé plusieurs domaines importants de la mécatronique, notamment la programmation, l'électronique, la conception assistée par ordinateur (CAO), la fabrication assistée par ordinateur (FAO) ainsi que l'impression 3D. Ce projet nous a permis de concevoir un système combinant des mécanismes mécaniques et un contrôle automatisé au sein d'une même plateforme fonctionnelle.

La conception du canon automatisé nous a amenés à développer différentes solutions afin de synchroniser plusieurs moteurs, gérer le système de rechargement et contrôler précisément le mécanisme de tir. Plusieurs ajustements mécaniques et logiciels ont également été nécessaires au cours des essais afin d'obtenir un fonctionnement plus stable, plus fluide et plus fiable du système.

Enfin, ce projet nous a permis d'appliquer concrètement différentes notions étudiées en mécatronique en réunissant programmation, mécanique, électronique, capteurs et automatisation dans un même système complet et fonctionnel.